LAPORAN PRAKTIKUM TERPADU

Docker Lab Studio

Pengujian Web + Terminal · Flask + MySQL + Docker Compose

Komponen	Isi
Aplikasi	Dashboard kontrol, aplikasi mahasiswa CRUD, Flask, MySQL 8.4, Docker Compose
Pengujian	T01–T10; individual dan Jalankan Semua Tes (T06 dilewati pada mode semua tes)
Bukti web	8 screenshot dashboard dan aplikasi mahasiswa
Bukti terminal	10 screenshot perintah dan output
Laporan	Langkah, ekspektasi, hasil contoh, waktu/durasi, log, ekspor JSON/CSV/HTML
Audit statis aktual	65 PASS, 0 FAIL, 5 BLOCKED runtime dari 70 pemeriksaan statis; runtime Docker belum dieksekusi di lingkungan penyusunan.

Catatan: status PASS pada screenshot run merupakan hasil simulasi yang diharapkan; Docker runtime belum dieksekusi di lingkungan penyusunan dokumen.

Ringkasan eksekutif

Paket ini menyatukan aplikasi web, dashboard pengujian, command center, screenshot web, screenshot terminal, transkrip, dan laporan akhir. Dashboard pada proyek Windows menggunakan PowerShell controller lokal yang menjalankan perintah Docker dan pemeriksaan HTTP aktual ketika dijalankan pada komputer pengguna.

Untuk bahan belajar, screenshot web dan terminal menunjukkan jalur sukses yang diharapkan dengan data contoh yang konsisten: Run ID 20261020-231455, 10 skenario, 9 PASS, 0 FAIL, dan 1 SKIPPED (T06). Keterangan simulasi ditempatkan pada caption, bukan di dalam screenshot.

Audit statis edisi perbaikan menjalankan 70 pemeriksaan: 65 PASS statis, 0 FAIL statis, dan 5 BLOCKED runtime. Docker CLI/Engine, Windows PowerShell, serta dependensi Flask/MySQL tidak tersedia pada lingkungan penyusunan, sehingga build dan integrasi runtime belum dapat diverifikasi di sini.

Koreksi dan validasi edisi perbaikan

Pada pemeriksaan ulang, beberapa titik logika dan konsistensi laporan diperbaiki agar pengujian lebih aman dan hasil contoh tidak menyatakan sesuatu yang tidak dibuktikan.

- Pembersihan data uji: setelah data berhasil dihapus, objek uji dikosongkan agar controller tidak mengirim permintaan hapus kedua kali. Jika tes gagal, controller mencoba cleanup ulang dan memberi peringatan bila data mungkin masih tertinggal.
- Pemulihan service: status pemulihan T04/T05 diatur sebelum perintah yang berpotensi menghentikan stack/database; blok finally mencoba memulihkan service dan mencatat peringatan jika pemulihan tidak terkonfirmasi.
- Validitas bukti HTTP: Invoke-WebRequest mengikuti redirect secara default. Bukti T03 kini menyatakan POST diikuti redirect dan memverifikasi record melalui GET, bukan mengklaim bahwa status 200 adalah status POST.
- Operasi destruktif: down -v membutuhkan konfirmasi di UI dan penanda konfirmasi eksplisit pada request controller. T06 tetap dikecualikan dari Jalankan Semua Tes.
- Ekspor laporan: CSV memakai BOM UTF-8 untuk kompatibilitas Excel; jika file JSON gagal disimpan, dashboard menampilkan peringatan dan tidak menampilkan tautan file yang tidak tersedia.
- Skrip Windows: reset memeriksa Docker Engine dan menyiapkan .env; skrip stop memeriksa ketersediaan Docker; reset tidak membuka dashboard bila controller belum aktif.
- Konsistensi materi: transkrip membedakan reset volume manual dari tes T06 otomatis. Screenshot dan laporan contoh tetap diberi caption simulasi, sedangkan audit statis dicatat terpisah.

Hasil audit statis: 65 PASS, 0 FAIL, 5 BLOCKED runtime dari 70 pemeriksaan. Status BLOCKED berarti runtime tidak tersedia di lingkungan penyusunan, bukan berarti pengujian runtime berhasil. Jalankan paket di Windows dengan Docker Desktop untuk memperoleh hasil aktual.

1. Arsitektur sistem

Komponen	Fungsi	Konfigurasi
Browser	Akses dashboard dan aplikasi mahasiswa	Dashboard 127.0.0.1:8765; aplikasi 127.0.0.1:5000
web (Flask)	Form CRUD mahasiswa dan endpoint health	Port host dibatasi ke 127.0.0.1:5000
db (MySQL 8.4)	Menyimpan data mahasiswa	Port 3306 internal; tidak dipublikasikan ke host
appnet	Network Compose	Flask terhubung ke MySQL melalui hostname db
dbdata	Named volume	Dipetakan ke /var/lib/mysql; bertahan pada down tanpa -v
Controller	Kontrol dan Test Center lokal	PowerShell HttpListener pada 127.0.0.1:8765

Alur utama: Browser → Flask web → MySQL db. Network appnet menyediakan DNS service db; volume dbdata mempertahankan data ketika container dihapus tanpa flag -v.

2. Menjalankan proyek di Windows

1. Ekstrak ZIP terpadu ke folder biasa, misalnya C:\DockerLabStudio. File jalankan-sistem.bat berada di root folder proyek.
2. Buka Docker Desktop dan tunggu Docker Engine siap. Build pertama membutuhkan koneksi internet untuk mengunduh image dan paket bila belum tersedia di cache.
3. Klik dua kali jalankan-sistem.bat. Skrip membuat .env jika belum ada, menyalakan controller lokal, menjalankan Compose, menunggu health, lalu membuka dashboard.
4. Buka dashboard http://127.0.0.1:8765/ dan aplikasi mahasiswa http://127.0.0.1:5000/.
5. Untuk semua tes non-destruktif, buka PowerShell pada folder proyek lalu jalankan .\uji-semua-tes.ps1 atau klik uji-semua-tes.bat.
6. Untuk tes individual gunakan .\uji-semua-tes.ps1 -TestId T03. Pilihan ID adalah T01–T10.
7. Jangan menjalankan T06 sebelum data penting dicadangkan dan bukti T04 disimpan. T06 memerlukan -AllowDestructive serta konfirmasi RESET.

3. Matriks pengujian T01–T10

ID	Tes	Apa yang diperiksa	Mode
T01	Prasyarat & konfigurasi	Docker Engine, .env, Compose config	Individual + Semua
T02	Build & startup service	Build, startup, /health, status container	Individual + Semua
T03	CRUD mahasiswa	Tambah, baca, hapus data uji	Individual + Semua
T04	Persistensi volume	Marker bertahan setelah down/up tanpa -v	Individual + Semua
T05	Pemulihan MySQL	Stop DB, degraded health, start dan pulih	Individual + Semua
T06	Reset volume database	down -v, buat volume baru, DB kosong	Individual saja; destruktif
T07	Network internal & DB	DNS service db dan koneksi MySQL internal	Individual + Semua
T08	Snapshot diagnostik	ps, logs, volume, network	Individual + Semua
T09	Port & storage	Port loopback, DB tanpa port host, named volume	Individual + Semua
T10	Validasi input	Input kosong dan NIM duplikat	Individual + Semua

Jalankan Semua Tes melewati T06 karena docker compose down -v menghapus seluruh data pada named volume proyek. Tes T04 dan T05 dapat membuat aplikasi/database tidak tersedia sementara; skrip melakukan up/start kembali sebagai pemulihan pengaman.

4. Detail prosedur dan perintah terminal

T01 — Prasyarat & konfigurasi

Tujuan: Memastikan Docker Engine, Docker Compose, .env, dan compose.yaml siap.

Langkah di web:

- Periksa versi Docker dan status controller pada dashboard.
- Klik Refresh untuk memperbarui status.

Perintah terminal / pemeriksaan:

```
docker --version
docker compose version
docker info --format '{{.ServerVersion}}'
Test-Path .env
docker compose config --quiet
```

Hasil yang diharapkan: Docker Engine merespons, .env tersedia, dan konfigurasi Compose valid.

T02 — Build & startup service

Tujuan: Membangun image web dan menjalankan Flask serta MySQL.

Langkah di web:

- Klik Up + Build di Command Center atau Jalankan T02 di Test Center.
- Tunggu Flask dan MySQL siap.

Perintah terminal / pemeriksaan:

```
docker compose up -d --build
docker compose ps --all
Invoke-RestMethod http://127.0.0.1:5000/health
```

Hasil yang diharapkan: Dua container berjalan; /health berstatus ok dan database connected.

T03 — Create, read & delete mahasiswa

Tujuan: Memverifikasi CRUD aplikasi Flask dan MySQL.

Langkah di web:

- Buka Aplikasi mahasiswa.
- Tambah data uji dengan NIM unik, pastikan muncul, lalu hapus.
- Atau klik Jalankan tes pada kartu T03.

Perintah terminal / pemeriksaan:

```
 $nim = "DLT20261020XXXX"
$body = @{ name = "Docker Lab Test CRUD"; nim = $nim }
Invoke-WebRequest -Uri "http://127.0.0.1:5000/students" -Method Post -Body $body
(Invoke-WebRequest "http://127.0.0.1:5000/").Content.Contains($nim)
```

Hasil yang diharapkan: Data dapat ditambahkan dan dibaca; setelah dihapus, NIM tidak lagi muncul.

T04 — Persistensi named volume

Tujuan: Membuktikan data MySQL bertahan setelah container dibuat ulang tanpa menghapus volume.

Langkah di web:

- Klik Jalankan T04.
- Tes membuat marker unik, menjalankan down/up tanpa -v, memverifikasi marker, lalu membersihkannya.

Perintah terminal / pemeriksaan:

```
$marker = "DLT20261020XXXX"
$body = @{ name = "Docker Lab Marker"; nim = $marker }
$null = Invoke-WebRequest -Uri "http://127.0.0.1:5000/students" -Method Post -Body $body -MaximumRedirection 5
-UseBasicParsing
docker compose down
docker compose up -d
$page = Invoke-WebRequest "http://127.0.0.1:5000/" -UseBasicParsing
$page.Content.Contains($marker)
$rowPattern = '(?is)<tr\b[^>]*>(?:(?!</tr>).)*?' + [regex]::Escape($marker) + '(?:(?!</tr>).)*?</tr>'
$row = [regex]::Match($page.Content, $rowPattern).Value
$idMatch = [regex]::Match($row, 'action="/students/(\d+)/delete"')
$studentId = [int]$idMatch.Groups[1].Value
$deleteUri = "http://127.0.0.1:5000/students/$studentId/delete"
$null = Invoke-WebRequest -Uri $deleteUri -Method Post -MaximumRedirection 5 -UseBasicParsing
$after = Invoke-WebRequest "http://127.0.0.1:5000/" -UseBasicParsing
$after.Content.Contains($marker)
```

Hasil yang diharapkan: Marker tetap ada setelah down/up; volume dbdata dipertahankan.

T05 — Pemulihan MySQL

Tujuan: Menguji respons Flask saat MySQL berhenti dan pulih.

Langkah di web:

- Klik Jalankan T05.
- Tes menghentikan db, memeriksa /live dan /health, menyalakan db kembali, lalu menunggu health pulih.

Perintah terminal / pemeriksaan:

```
docker compose stop db
Invoke-RestMethod http://127.0.0.1:5000/live
Invoke-RestMethod http://127.0.0.1:5000/health
docker compose start db
```

Hasil yang diharapkan: Flask tetap hidup; /health menjadi degraded ketika DB berhenti dan kembali ok setelah DB pulih.

T06 — Reset volume database (destruktif)

Tujuan: Membuktikan down -v menghapus named volume dan data di dalamnya.

Langkah di web:

- Jangan jalankan T06 melalui Jalankan Semua Tes.
- Backup data dan simpan bukti T04 terlebih dahulu.
- Jalankan tes individual T06 dan konfirmasi eksplisit.

Perintah terminal / pemeriksaan:

```
docker compose down -v
docker compose up -d --build
Invoke-RestMethod http://127.0.0.1:5000/health
```

Hasil yang diharapkan: Volume dibuat ulang dan database baru kosong. Data lama pada volume proyek akan hilang.

PERINGATAN: Tindakan ini menghapus data dalam volume proyek. Jalankan hanya setelah backup dan konfirmasi eksplisit.

T07 — Network internal & koneksi DB

Tujuan: Memastikan web menemukan MySQL melalui hostname db di network Compose.

Langkah di web:

- Klik Jalankan T07 setelah stack sehat.
- Tes menjalankan DNS lookup dan koneksi MySQL dari container web.

Perintah terminal / pemeriksaan:

```
docker compose exec -T web python -c "import socket; print(socket.gethostbyname('db'))"
docker compose exec -T web python -c "import os,mysql.connector;
c=mysql.connector.connect(host=os.getenv('DB_HOST','db'),user=os.getenv('DB_USER'),password=os.getenv('DB_PASSWORD'),d
atabase=os.getenv('DB_NAME')); c.close(); print('MYSQL_CONNECT_OK')"
```

Hasil yang diharapkan: Hostname db ter-resolve dan koneksi MySQL internal berhasil.

T08 — Snapshot diagnostik

Tujuan: Mengumpulkan bukti status container, log, volume, dan network.

Langkah di web:

- Klik Jalankan T08.
- Bandingkan output di Command Center, lalu buka Reports & evidence.

Perintah terminal / pemeriksaan:

```
docker compose ps --all
docker compose logs --tail 80 --no-color
docker volume ls --filter label=com.docker.compose.project=flask-mysql-docker-lab
docker network ls --filter label=com.docker.compose.project=flask-mysql-docker-lab
```

Hasil yang diharapkan: Status, log, volume, dan network berhasil dikumpulkan.

T09 — Port & storage configuration

Tujuan: Memastikan web hanya dipublikasikan ke loopback, DB tidak memiliki port host, dan data memakai named volume.

Langkah di web:

- Klik Jalankan T09.
- Periksa compose.yaml bila diperlukan; tes tidak menghentikan service.

Perintah terminal / pemeriksaan:

```
docker compose config
Select-String -Path .\compose.yaml -Pattern '127.0.0.1:5000:5000','dbdata:/var/lib/mysql'
```

Hasil yang diharapkan: Web terikat pada 127.0.0.1:5000; MySQL tidak dipublikasikan ke host; dbdata dipasang ke /var/lib/mysql.

T10 — Validasi input & NIM duplikat

Tujuan: Memastikan server menolak input kosong dan NIM yang sudah digunakan.

Langkah di web:

- Klik Jalankan T10.
- Tes membuat satu record, mengirim NIM sama, mengirim form kosong, lalu membersihkan data uji.

Perintah terminal / pemeriksaan:

```
$body = @{ name = "Docker Lab Duplicate Test"; nim = $nim }
Invoke-WebRequest -Uri "http://127.0.0.1:5000/students" -Method Post -Body $body
Invoke-WebRequest -Uri "http://127.0.0.1:5000/students" -Method Post -Body @{ name=""; nim="" }
```

Hasil yang diharapkan: Input kosong dan NIM duplikat ditolak; record uji dibersihkan.

5. Hasil contoh yang diharapkan

ID	Status contoh	Durasi contoh	Ringkasan
T01	PASS	245 ms	Engine dan konfigurasi Compose valid
T02	PASS	24.1 s	Image build dan service sehat
T03	PASS	804 ms	Create/read/delete berhasil
T04	PASS	18.9 s	Marker bertahan setelah down/up
T05	PASS	13.3 s	MySQL pulih setelah stop/start
T06	SKIPPED	0 ms	Dilewati karena destruktif
T07	PASS	530 ms	DNS dan koneksi DB berhasil
T08	PASS	912 ms	Bukti diagnostik terkumpul
T09	PASS	3 ms	Port dan volume valid
T10	PASS	1.17 s	Validasi input berhasil

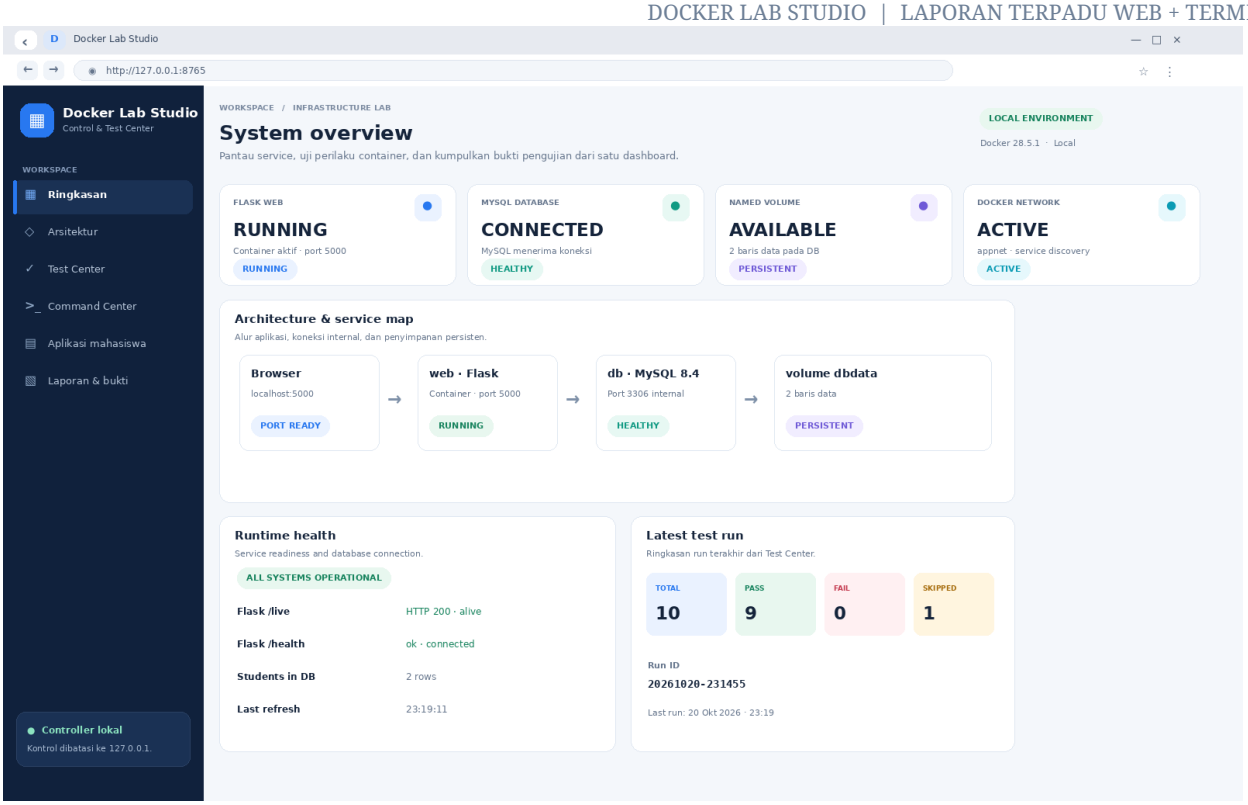
Angka durasi dan Run ID merupakan contoh untuk menunjukkan bentuk laporan; angka aktual dihitung saat tes dijalankan di Windows.

6. Laporan dan ekspor

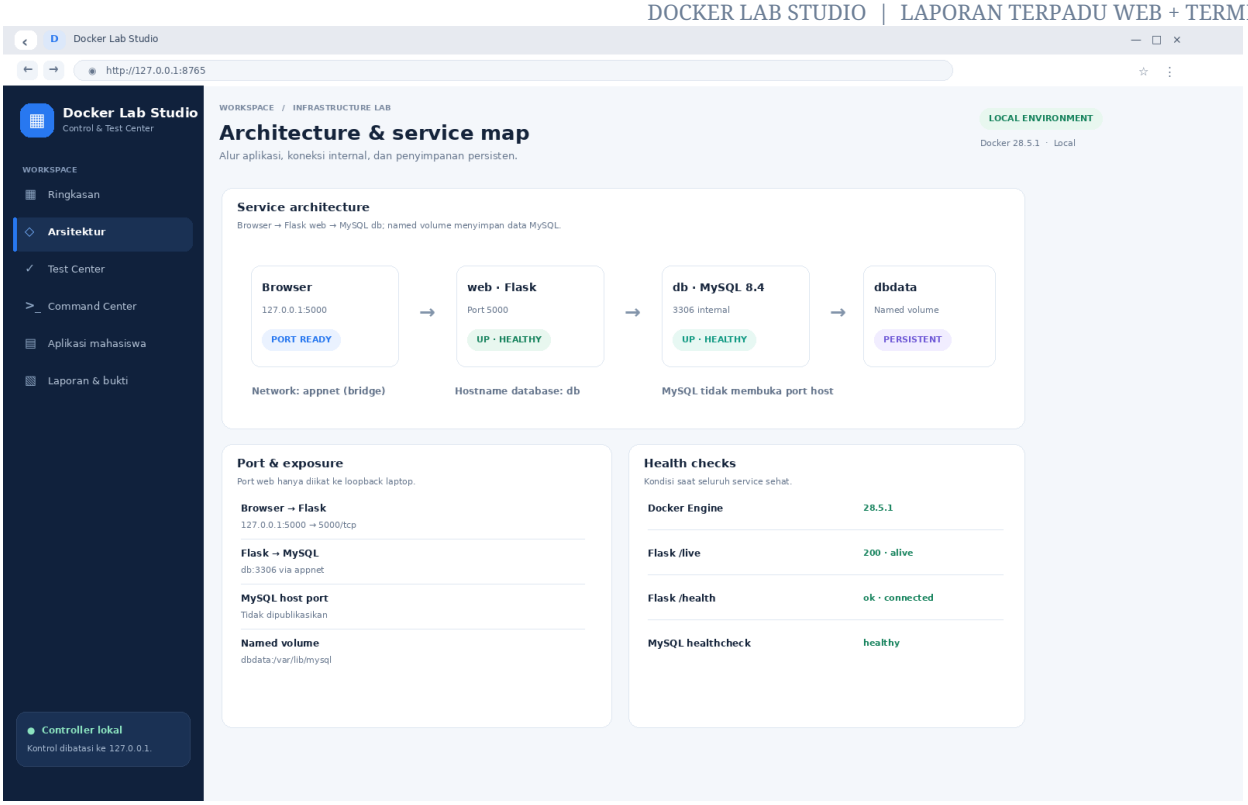
- Setiap hasil memuat status, waktu mulai/selesai, durasi, langkah, ekspektasi, hasil aktual, bukti, log, dan error jika ada.
- File JSON setiap run disimpan di reports\test-report-<runId>.json.
- Menu Reports & evidence menyediakan ekspor HTML, CSV, dan JSON.
- Command Center memperlihatkan output perintah Docker dan log controller; uji-semua-tes.ps1 mencetak rekap di PowerShell.
- Screenshot di laporan merupakan bahan belajar dan bukan pengganti bukti aktual jika instruksi tugas mensyaratkan screenshot dari laptop.

Bagian I — Bukti tampilan web

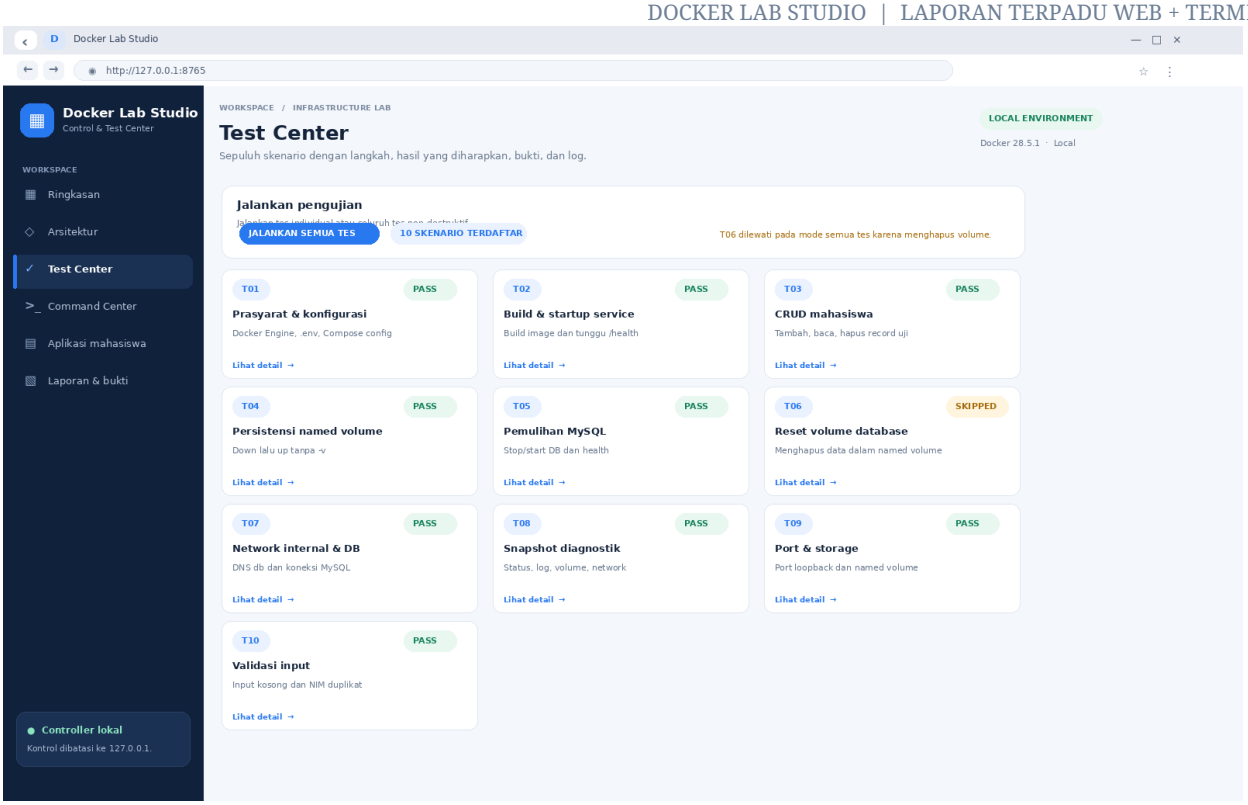
Tampilan berikut mengikuti layout dashboard pada proyek: status service, peta arsitektur, Test Center, hasil tes, Command Center, aplikasi mahasiswa, dan ekspor laporan.



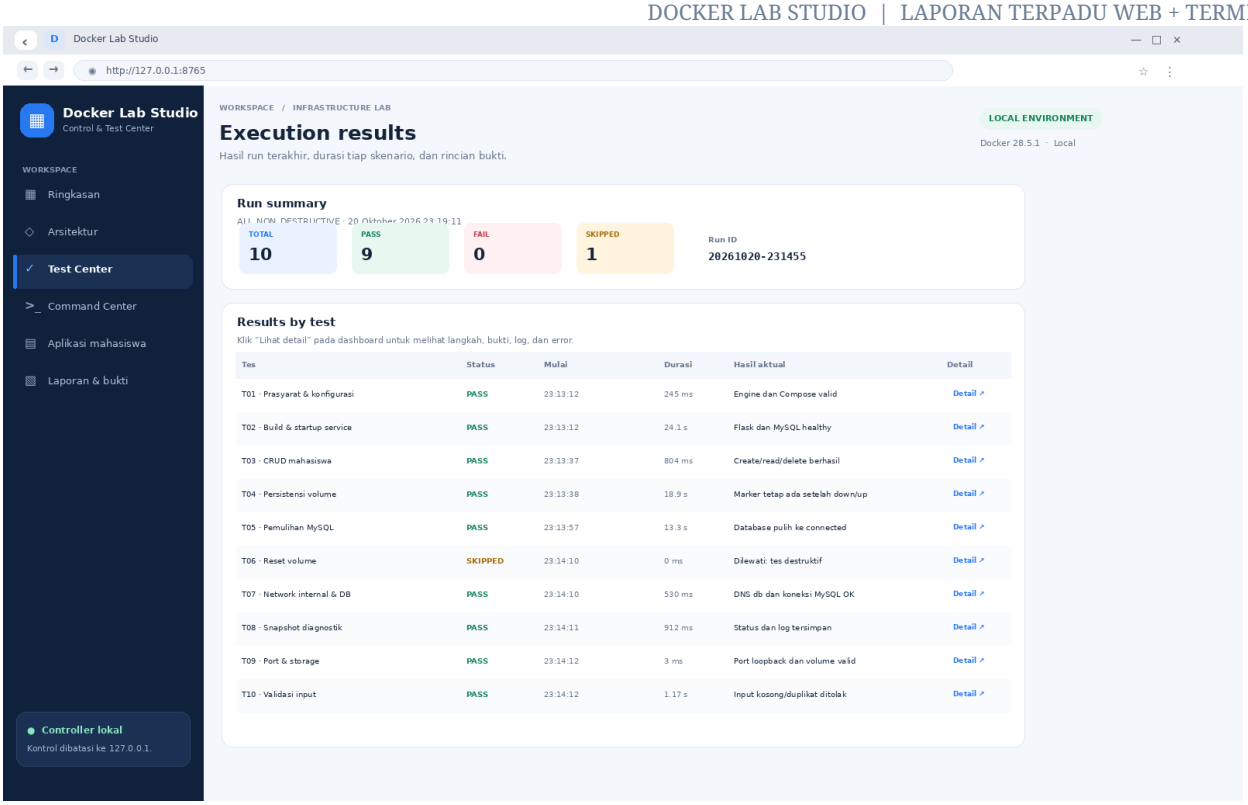
Gambar 1.1. Dashboard ringkasan status Flask, MySQL, volume, network, dan hasil tes terakhir. Gambar ini merupakan simulasi visual untuk bahan belajar.



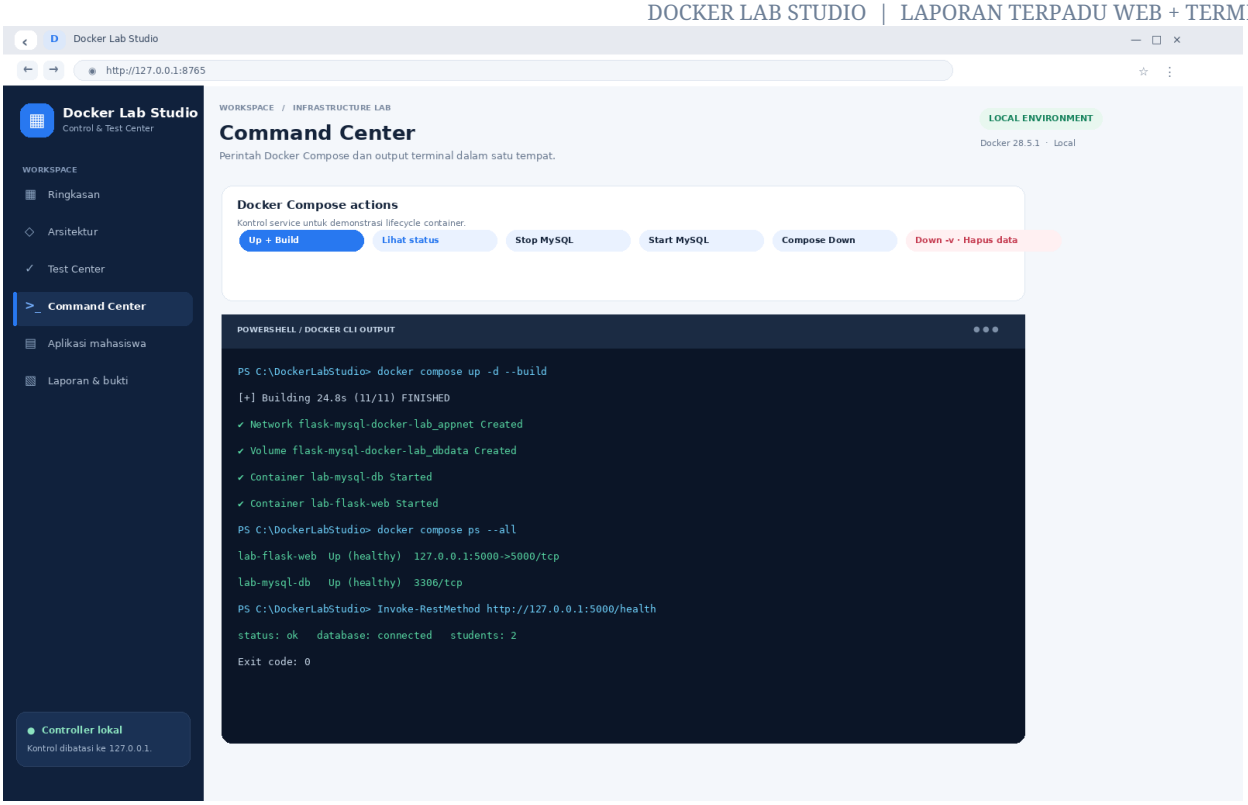
Gambar 1.2. Peta arsitektur Browser → Flask → MySQL, network appnet, dan named volume dbdata. Gambar ini merupakan simulasi visual untuk bahan belajar.



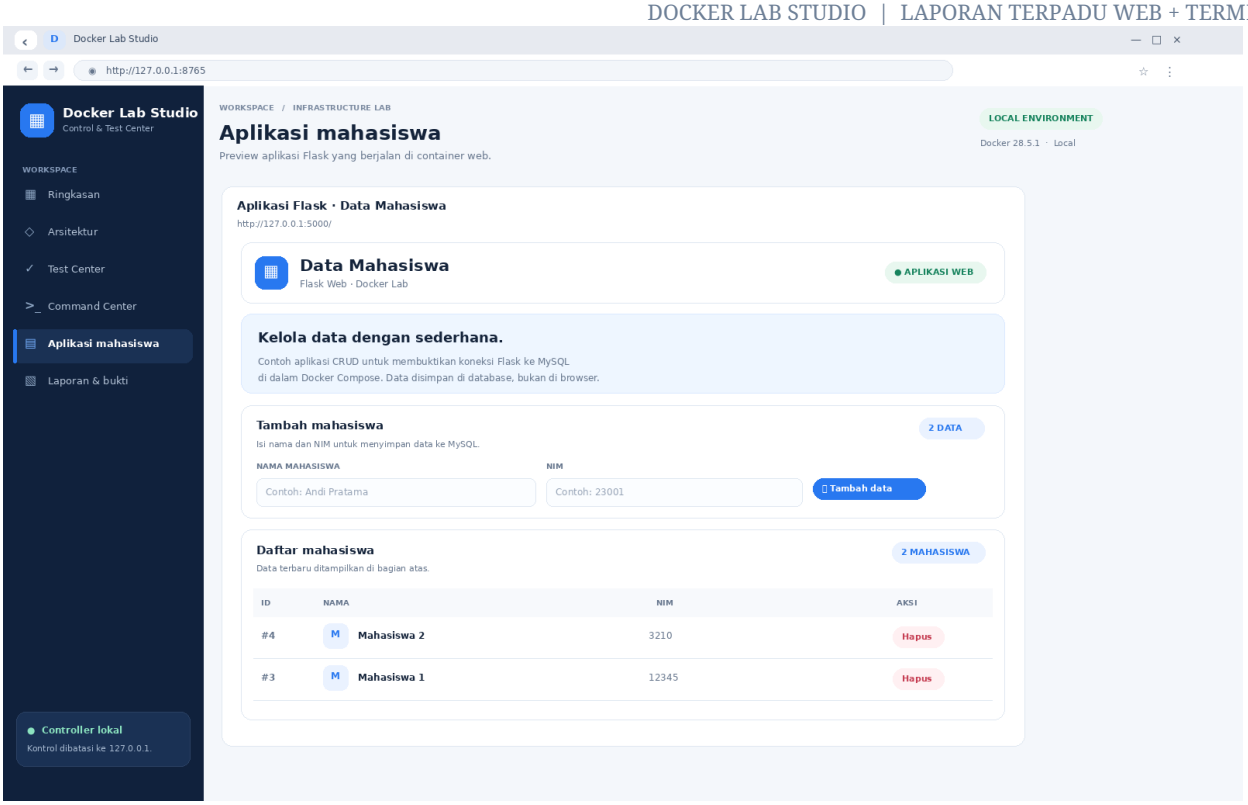
Gambar 1.3. Test Center menampilkan sepuluh skenario, tombol Jalankan Semua Tes, status setiap tes, dan peringatan untuk tes destruktif. Gambar ini merupakan simulasi visual untuk bahan belajar.



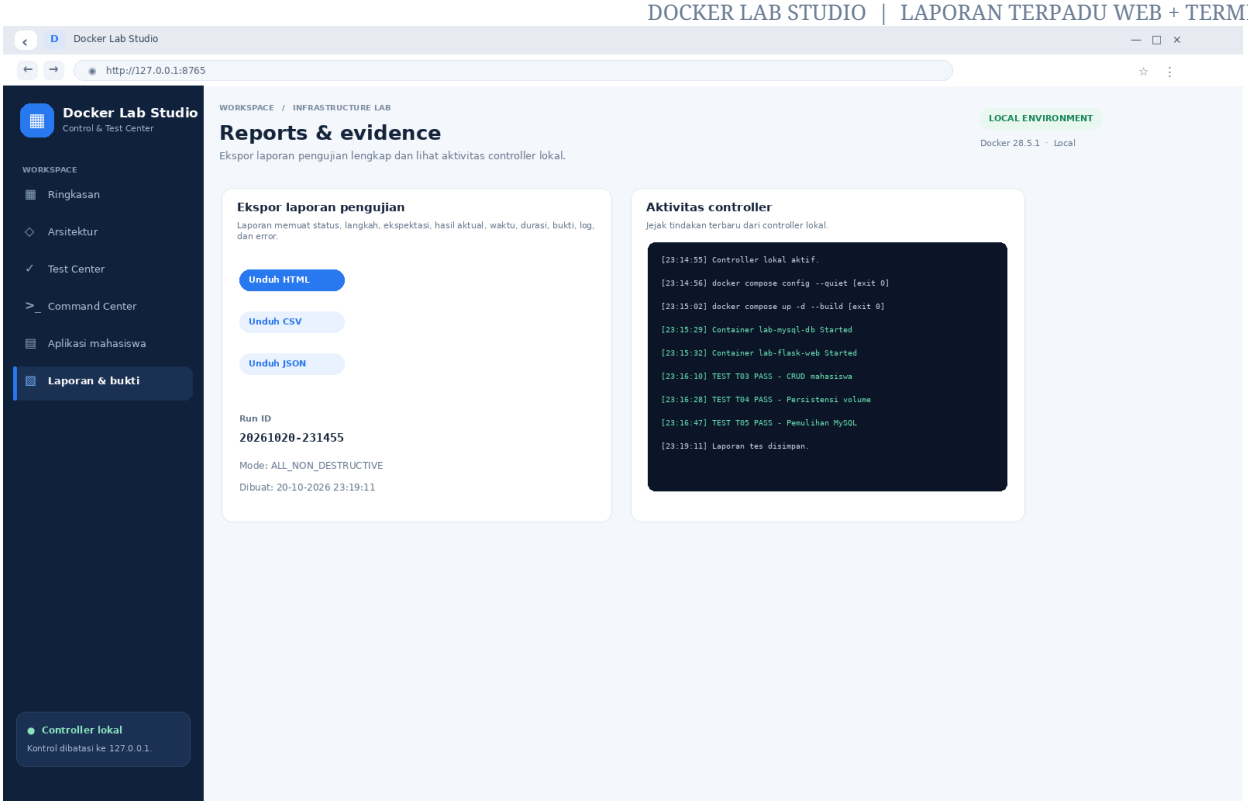
Gambar 1.4. Execution results menampilkan Run ID, ringkasan PASS/FAIL/SKIPPED, durasi, dan hasil aktual setiap tes. Gambar ini merupakan simulasi visual untuk bahan belajar.



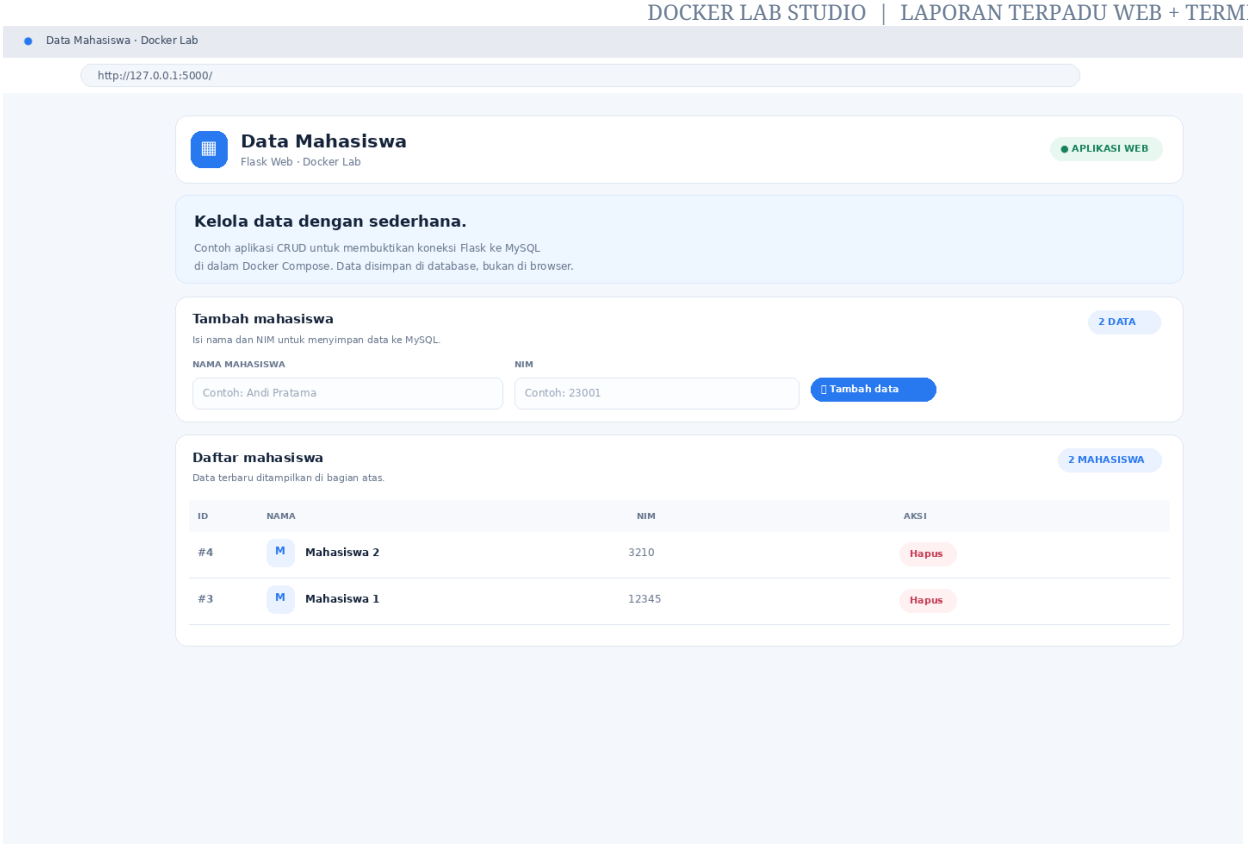
Gambar 1.5. Command Center menampilkan tombol perintah Docker Compose dan output terminal build, status container, serta health endpoint. Gambar ini merupakan simulasi visual untuk bahan belajar.



Gambar 1.6. Dashboard menampilkan preview aplikasi Flask untuk menambah dan melihat data mahasiswa. Gambar ini merupakan simulasi visual untuk bahan belajar.



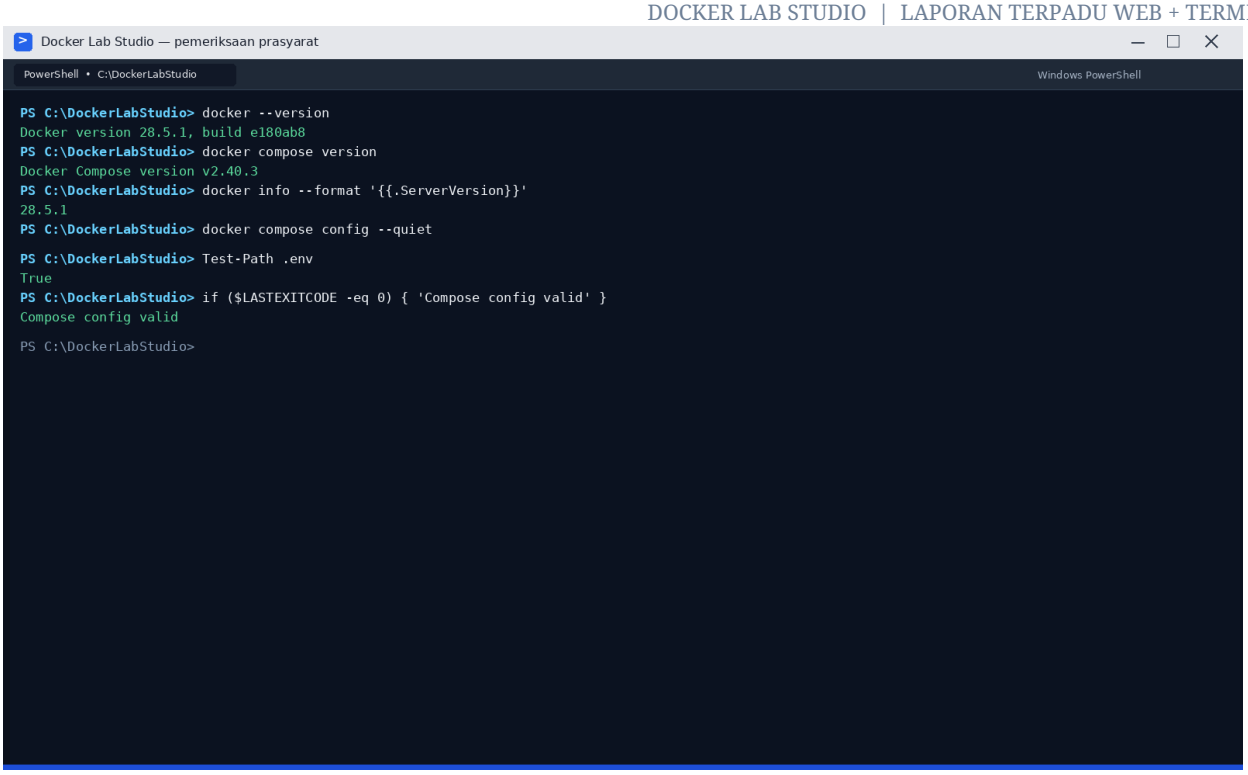
Gambar 1.7. Reports & evidence menampilkan ekspor HTML, CSV, JSON, Run ID, dan aktivitas controller. Gambar ini merupakan simulasi visual untuk bahan belajar.



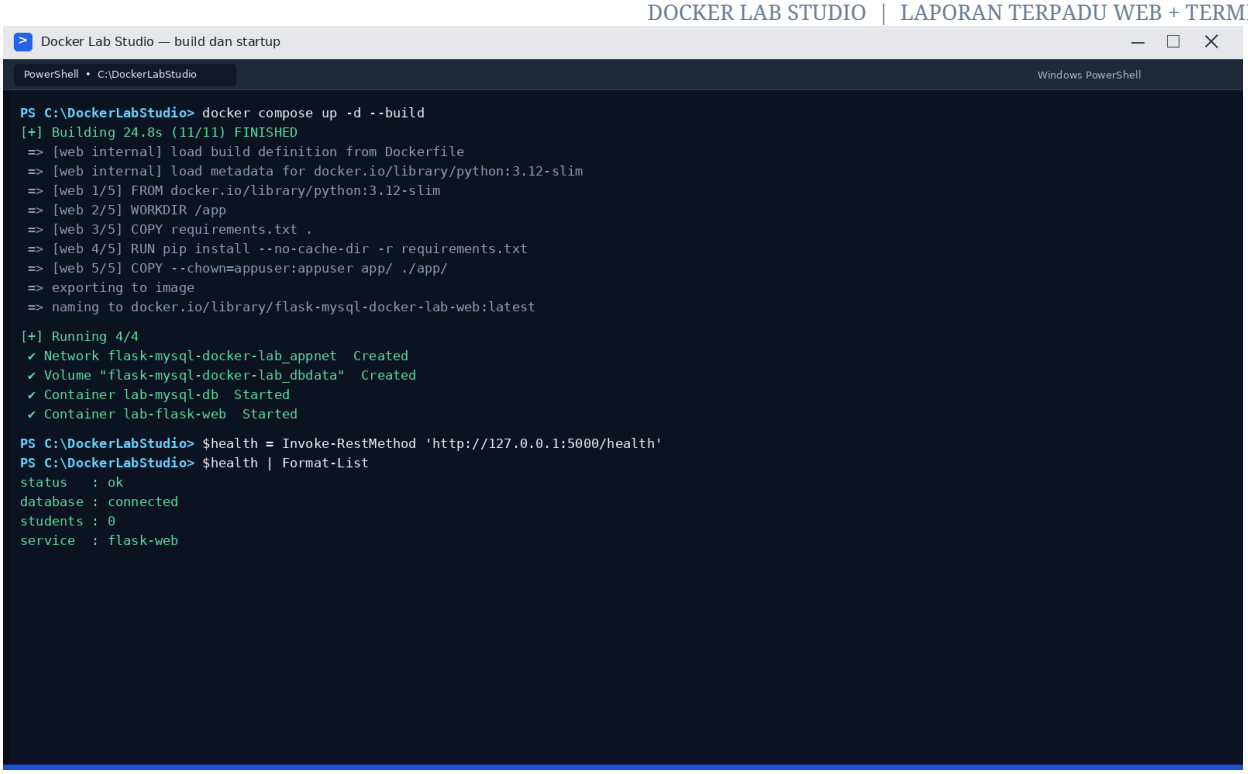
Gambar 1.8. Aplikasi mahasiswa berdiri sendiri pada localhost:5000 dengan formulir tambah data dan daftar mahasiswa. Gambar ini merupakan simulasi visual untuk bahan belajar.

Bagian II — Bukti terminal

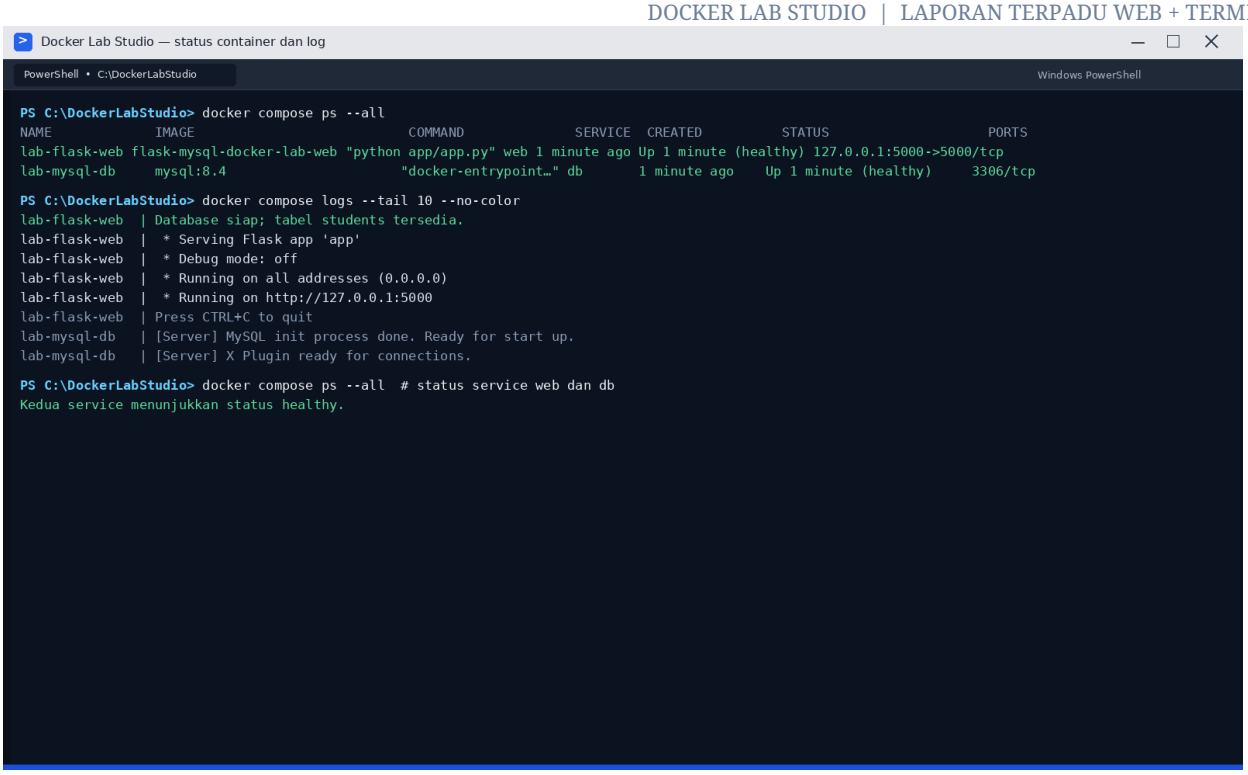
Terminal berikut menunjukkan perintah dan bentuk output yang perlu dibandingkan dengan hasil PowerShell di laptop.



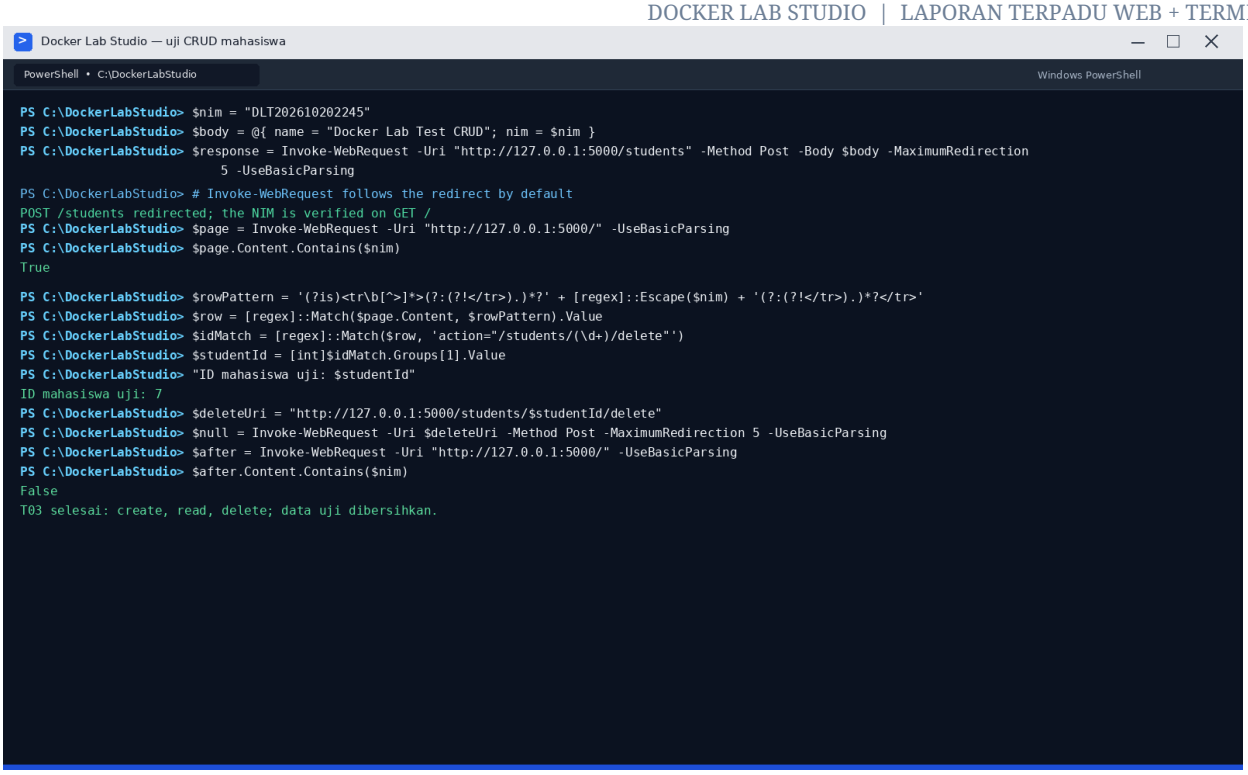
Gambar 2.1. Pemeriksaan versi Docker, Compose, Engine, file .env, dan validitas compose.yaml. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



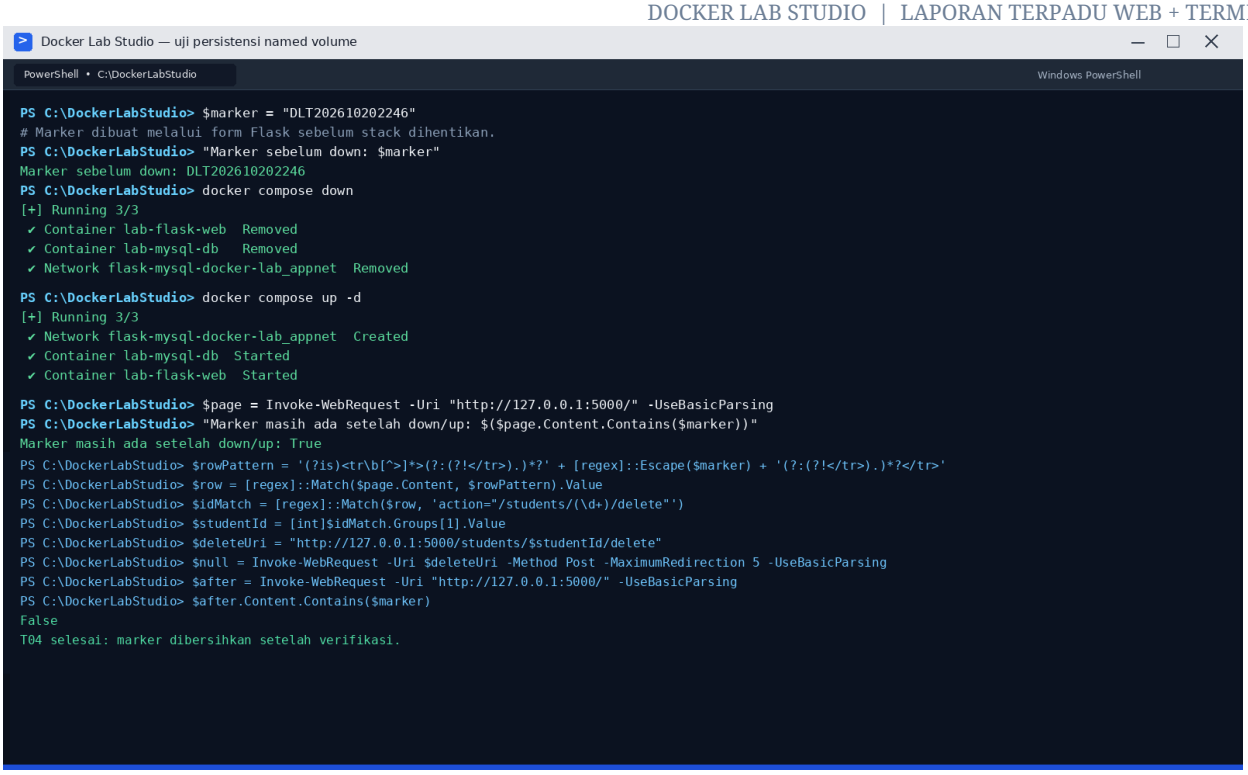
Gambar 2.2. Proses docker compose up -d --build, build image Flask, pembuatan network dan volume, serta startup container. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



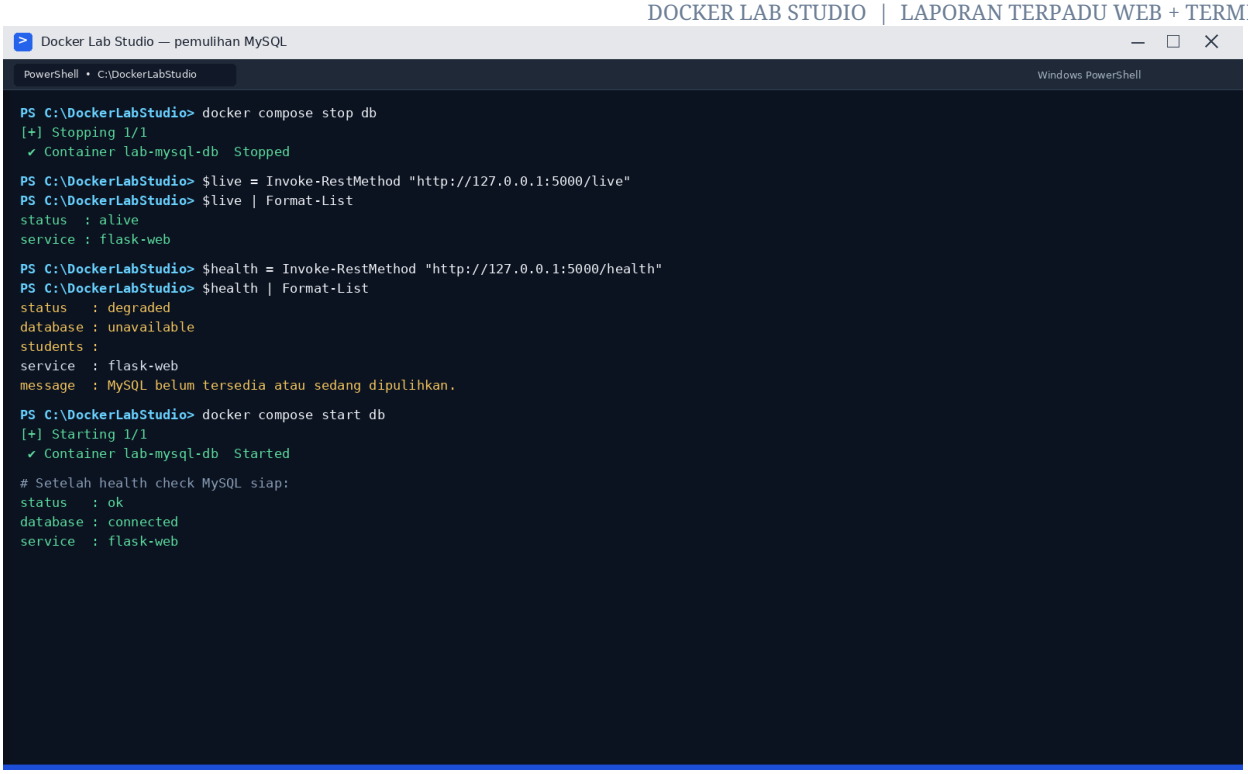
Gambar 2.3. Pemeriksaan docker compose ps --all dan log Flask/MySQL setelah startup. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



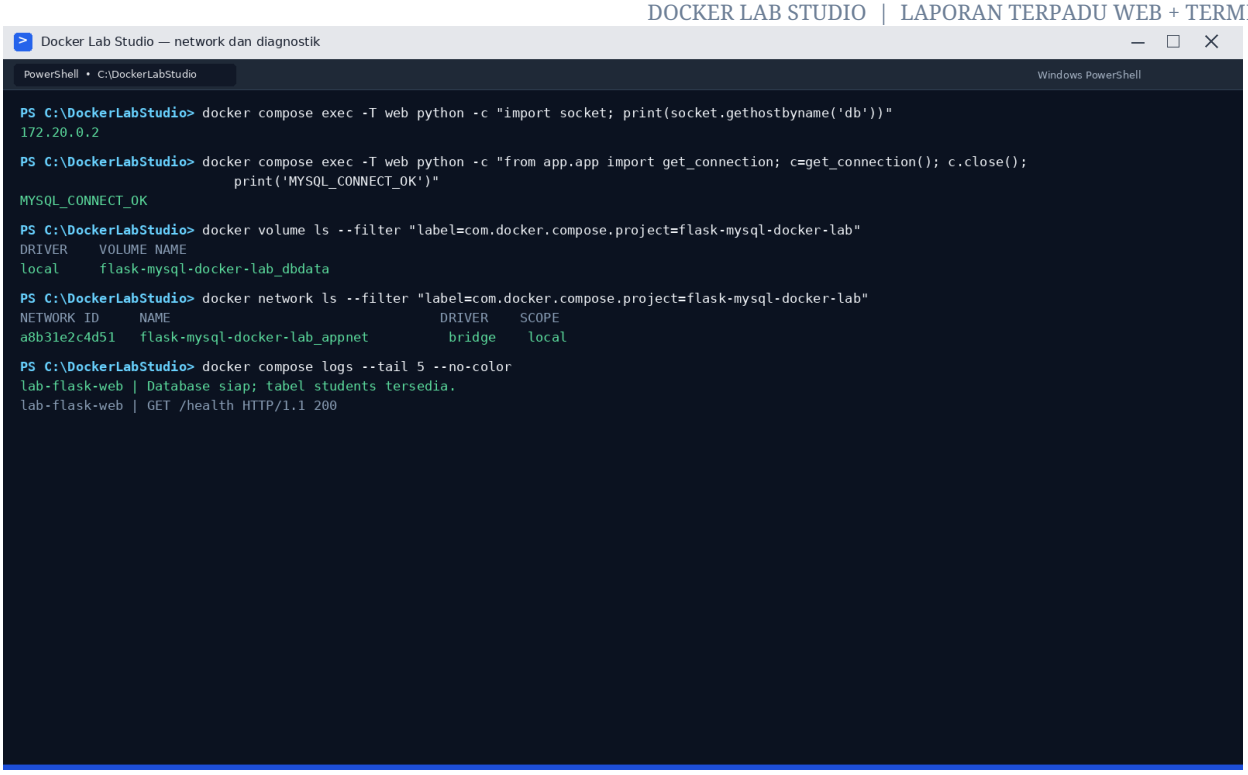
Gambar 2.4. Pengujian Create, Read, dan Delete data mahasiswa melalui HTTP, termasuk verifikasi NIM dan pembersihan data uji. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



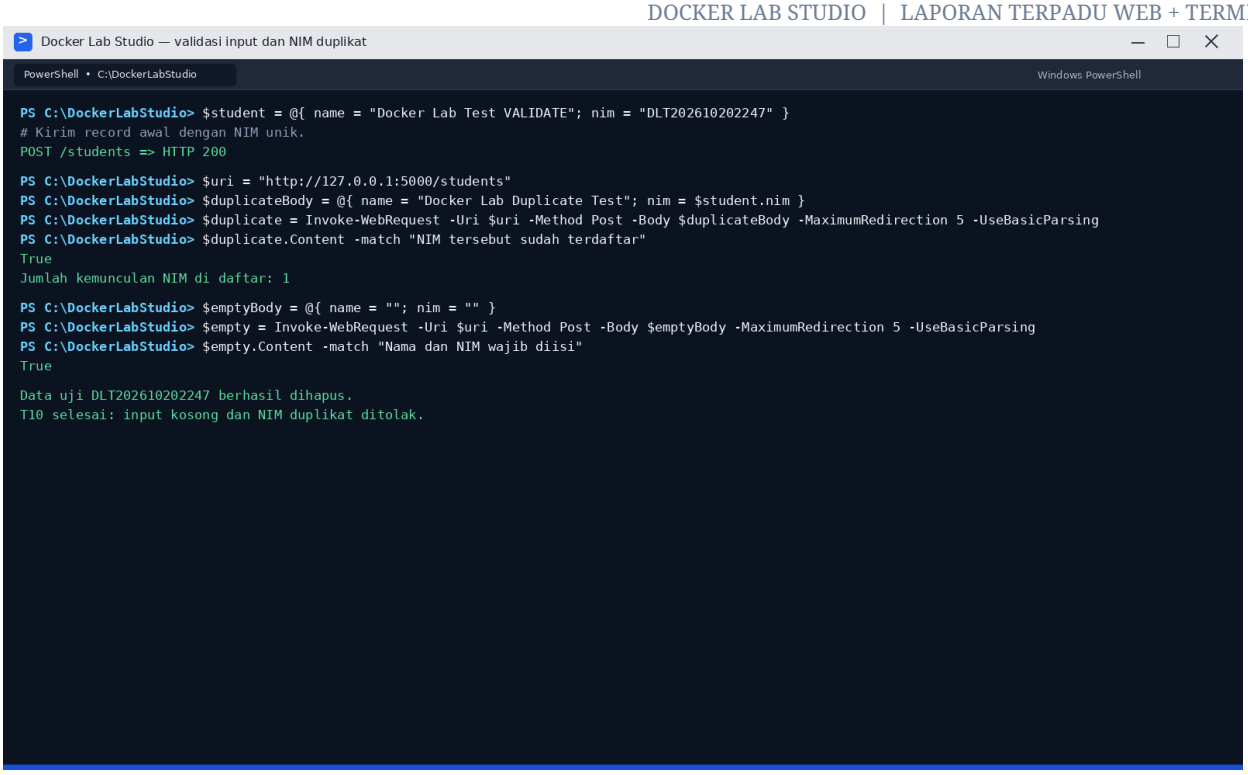
Gambar 2.5. Pengujian persistensi marker setelah docker compose down dan docker compose up tanpa -v. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



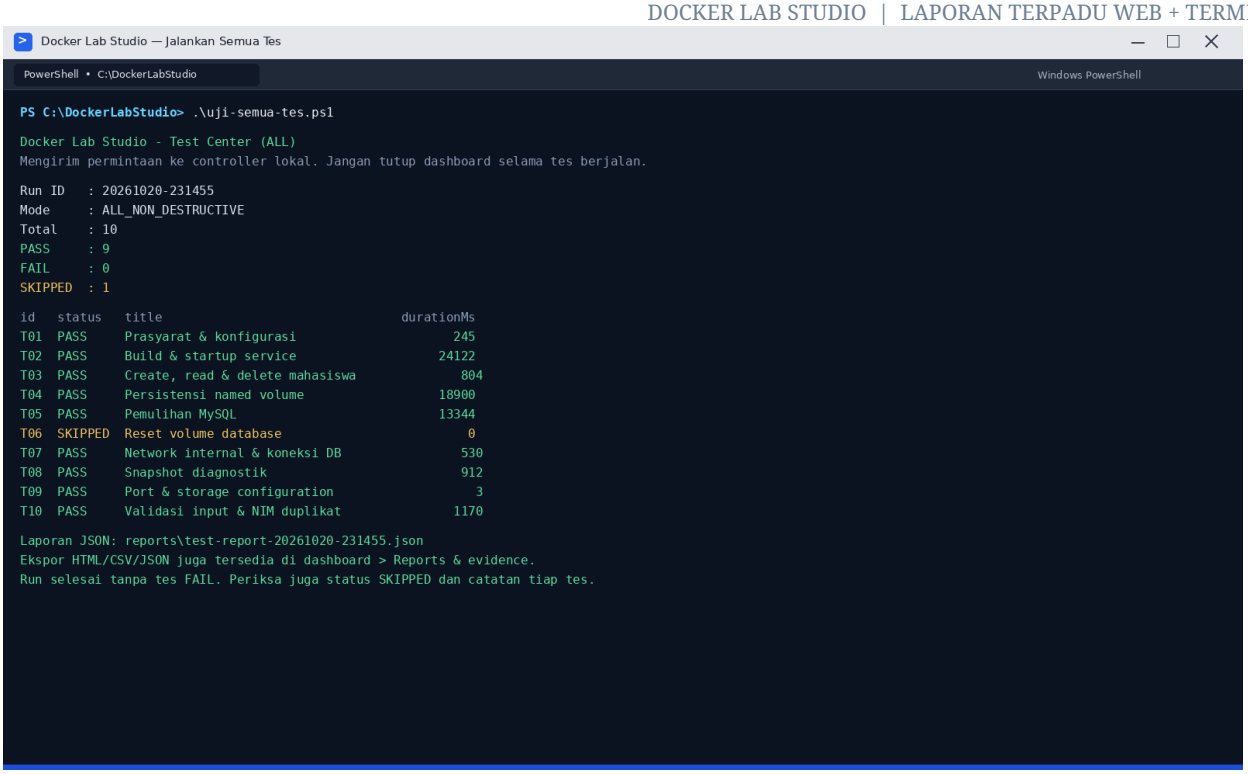
Gambar 2.6. Penghentian sementara MySQL, pemeriksaan /live dan /health, lalu pemulihan database. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



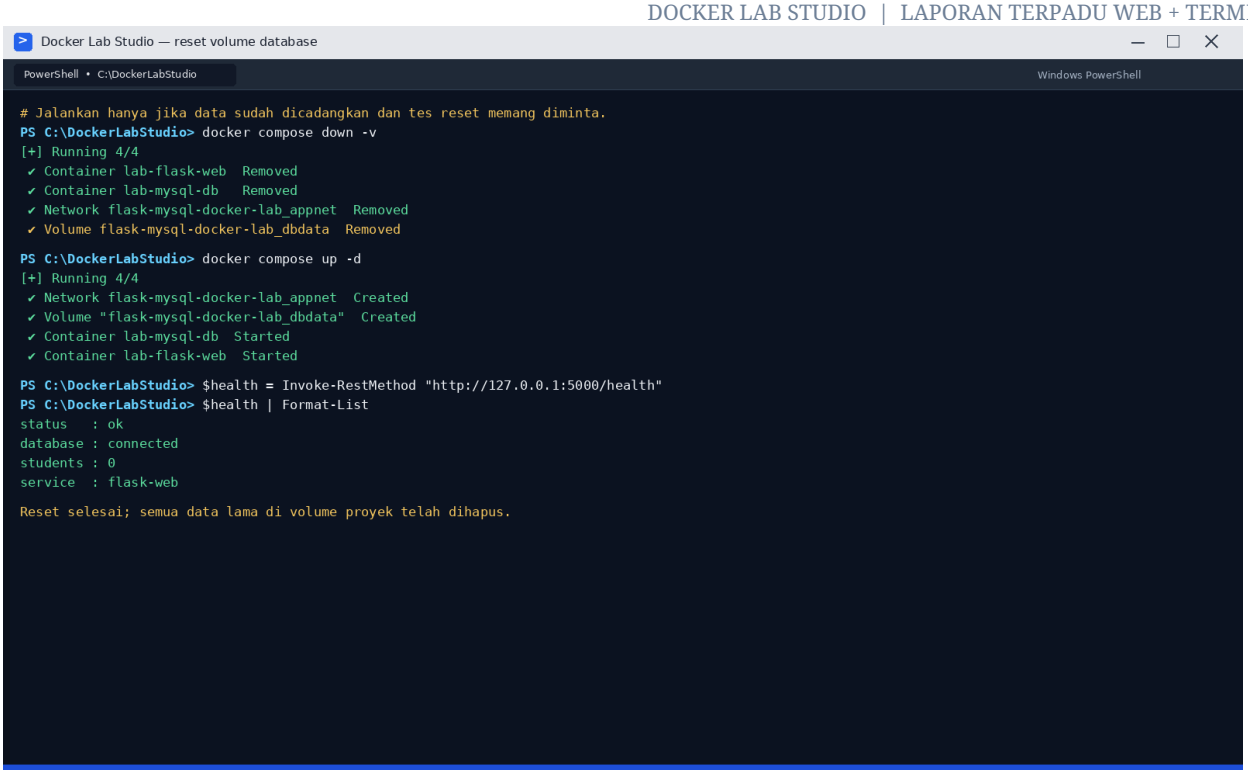
Gambar 2.7. Pengujian resolusi hostname db, koneksi MySQL dari container web, dan pemeriksaan volume/network. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



Gambar 2.8. Pengujian penolakan input kosong dan NIM duplikat, kemudian pembersihan data uji. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



Gambar 2.9. Ringkasan tes T01–T10 dari skrip terminal, dengan sembilan PASS dan satu SKIPPED untuk T06. Gambar ini merupakan simulasi output terminal untuk bahan belajar.



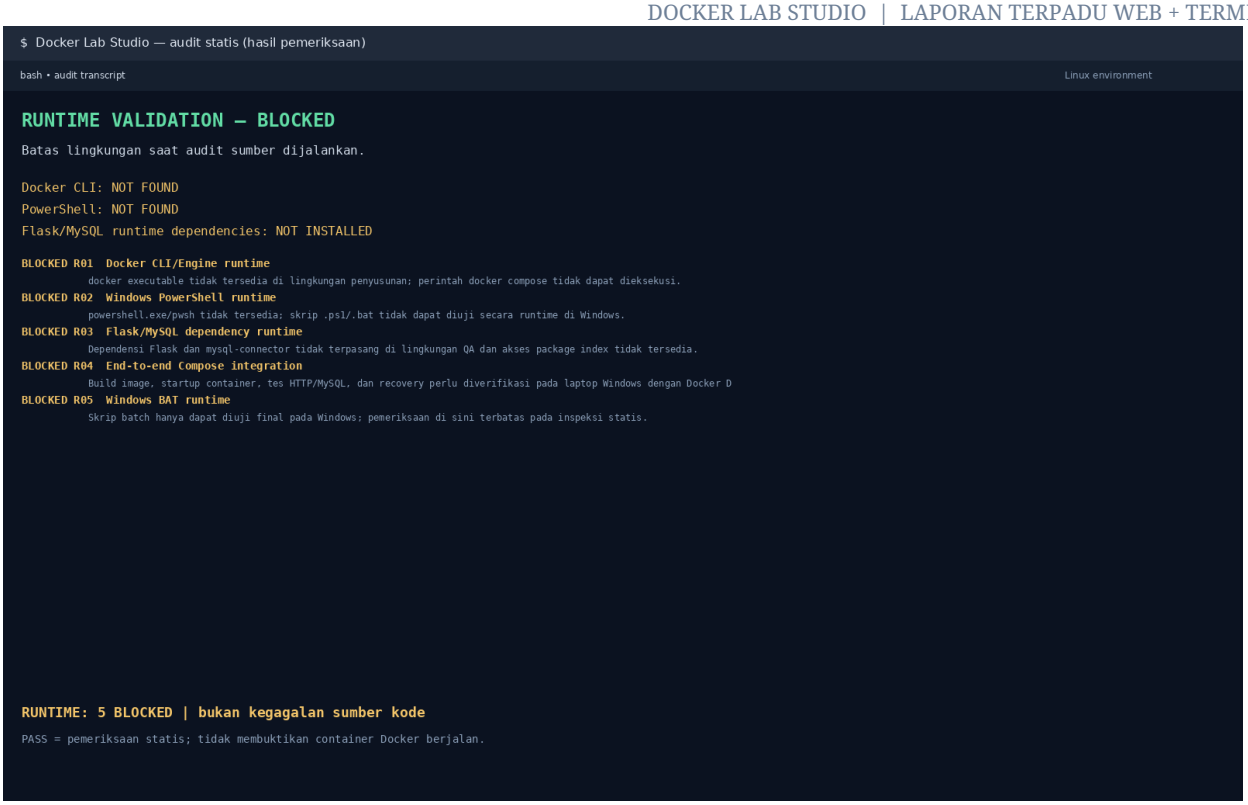
Gambar 2.10. Contoh reset volume manual (opsional), penghapusan named volume, dan pemeriksaan database baru kosong. Gambar ini merupakan simulasi output terminal untuk bahan belajar.

Bagian III — Audit statis aktual

Dua gambar berikut merangkum pemeriksaan pra-eksekusi yang benar-benar dilakukan pada berkas sumber di lingkungan penyusunan paket. Pemeriksaan statis tidak membuktikan bahwa container Docker telah berjalan.



Gambar 3.1. Ringkasan pemeriksaan statis aktual pada berkas dan konfigurasi.



Gambar 3.2. Keterbatasan runtime aktual pada lingkungan penyusunan paket.

Lampiran A — Transkrip terminal lengkap

Transkrip berikut adalah contoh output yang diharapkan. Cocokkan dengan output aktual; versi, durasi build, ID, IP container, dan log dapat berbeda.

```
DOCKER LAB STUDIO — TRANSKRIP TERMINAL LENGKAP
================================================
Jenis: CONTOH OUTPUT YANG DIHARAPKAN / SIMULASI
Bukan hasil runtime Docker aktual. Gunakan untuk memahami urutan dan membandingkan dengan terminal laptop.

A. PRASYARAT (Gambar 2.1)
PS C:\DockerLabStudio> docker --version
Docker version 28.5.1, build e180ab8
PS C:\DockerLabStudio> docker compose version
Docker Compose version v2.40.3
PS C:\DockerLabStudio> docker info --format '{{.ServerVersion}}'
28.5.1
PS C:\DockerLabStudio> Test-Path .env
True
PS C:\DockerLabStudio> docker compose config --quiet
PS C:\DockerLabStudio> if ($LASTEXITCODE -eq 0) { 'Compose config valid' }
Compose config valid

B. BUILD IMAGE DAN STARTUP (Gambar 2.2)
PS C:\DockerLabStudio> docker compose up -d --build
[+] Building 24.8s (11/11) FINISHED
 => [web internal] load build definition from Dockerfile
 => [web internal] load metadata for docker.io/library/python:3.12-slim
 => [web 1/5] FROM docker.io/library/python:3.12-slim
 => [web 2/5] WORKDIR /app
 => [web 3/5] COPY requirements.txt .
 => [web 4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [web 5/5] COPY --chown=appuser:appuser app/ ./app/
 => exporting to image
 => naming to docker.io/library/flask-mysql-docker-lab-web:latest
[+] Running 4/4
✔ Network flask-mysql-docker-lab_appnet  Created
✔ Volume "flask-mysql-docker-lab_dbdata"  Created
✔ Container lab-mysql-db  Started
✔ Container lab-flask-web  Started
PS C:\DockerLabStudio> Invoke-RestMethod 'http://127.0.0.1:5000/health' | Format-List
status   : ok
database : connected
students : 0
service  : flask-web

C. STATUS DAN LOG (Gambar 2.3)
PS C:\DockerLabStudio> docker compose ps --all
NAME           IMAGE                       SERVICE  STATUS       PORTS
lab-flask-web    flask-mysql-docker-lab-web     web      Up (healthy)  127.0.0.1:5000->5000/tcp
lab-mysql-db     mysql:8.4                   db       Up (healthy)  3306/tcp
PS C:\DockerLabStudio> docker compose logs --tail 10 --no-color
lab-flask-web | Database siap; tabel students tersedia.
lab-flask-web | * Serving Flask app 'app'
lab-flask-web | * Debug mode: off
lab-flask-web | * Running on all addresses (0.0.0.0)
lab-flask-web | * Running on http://127.0.0.1:5000
lab-mysql-db  | [Server] MySQL init process done. Ready for start up.

D. CRUD MAHASISWA (Gambar 2.4)
PS C:\DockerLabStudio> $nim = "DLT202610202245"
PS C:\DockerLabStudio> $body = @{ name = "Docker Lab Test CRUD"; nim = $nim }
```

```
PS C:\DockerLabStudio> $response = Invoke-WebRequest -Uri "http://127.0.0.1:5000/students" -Method Post -Body $body
-MaximumRedirection 5 -UseBasicParsing
PS C:\DockerLabStudio> $page = Invoke-WebRequest -Uri "http://127.0.0.1:5000/" -UseBasicParsing
PS C:\DockerLabStudio> $page.Content.Contains($nim)
True
PS C:\DockerLabStudio> # Ambil ID dari baris HTML yang memuat NIM, jangan hardcode ID.
PS C:\DockerLabStudio> $rowPattern = '(?is)<tr\b[^>]*>(?:(?!</tr>).)*?' + [regex]::Escape($nim) + '(?:(?!</tr>).)*?</tr>'
PS C:\DockerLabStudio> $row = [regex]::Match($page.Content, $rowPattern).Value
PS C:\DockerLabStudio> $idMatch = [regex]::Match($row, 'action="/students/(\d+)/delete"')
PS C:\DockerLabStudio> $studentId = [int]$idMatch.Groups[1].Value
PS C:\DockerLabStudio> $deleteUri = "http://127.0.0.1:5000/students/$studentId/delete"
PS C:\DockerLabStudio> $null = Invoke-WebRequest -Uri $deleteUri -Method Post -MaximumRedirection 5 -UseBasicParsing
PS C:\DockerLabStudio> $after = Invoke-WebRequest -Uri "http://127.0.0.1:5000/" -UseBasicParsing
PS C:\DockerLabStudio> $after.Content.Contains($nim)
False
```

E. PERSISTENSI VOLUME (Gambar 2.5 / T04)

Tentukan NIM marker yang unik, lalu tambahkan record dengan NIM yang sama melalui aplikasi web.

```
PS C:\DockerLabStudio> $marker = "DLT202610202246"
```

Pastikan record dengan NIM tersebut tersimpan sebelum menghentikan stack.

```
PS C:\DockerLabStudio> docker compose down
[+] Running 3/3
 ✔ Container lab-flask-web  Removed
 ✔ Container lab-mysql-db   Removed
 ✔ Network flask-mysql-docker-lab_appnet  Removed
Volume dbdata dipertahankan.
PS C:\DockerLabStudio> docker compose up -d
[+] Running 3/3
 ✔ Network flask-mysql-docker-lab_appnet  Created
 ✔ Container lab-mysql-db  Started
 ✔ Container lab-flask-web Started
PS C:\DockerLabStudio> $page = Invoke-WebRequest 'http://127.0.0.1:5000/' -UseBasicParsing
PS C:\DockerLabStudio> $page.Content.Contains($marker)
True
PS C:\DockerLabStudio> # Cari ID dari baris yang memuat marker, lalu hapus data uji setelah verifikasi.
PS C:\DockerLabStudio> $rowPattern = '(?is)<tr\b[^>]*>(?:(?!</tr>).)*?' + [regex]::Escape($marker) + '(?:(?!</tr>).)*?</tr>'
PS C:\DockerLabStudio> $row = [regex]::Match($page.Content, $rowPattern).Value
PS C:\DockerLabStudio> $idMatch = [regex]::Match($row, 'action="/students/(\d+)/delete"')
PS C:\DockerLabStudio> $studentId = [int]$idMatch.Groups[1].Value
PS C:\DockerLabStudio> $deleteUri = "http://127.0.0.1:5000/students/$studentId/delete"
PS C:\DockerLabStudio> $null = Invoke-WebRequest -Uri $deleteUri -Method Post -MaximumRedirection 5 -UseBasicParsing
PS C:\DockerLabStudio> $after = Invoke-WebRequest 'http://127.0.0.1:5000/' -UseBasicParsing
PS C:\DockerLabStudio> $after.Content.Contains($marker)
False
```

F. PEMULIHAN MYSQL (Gambar 2.6 / T05)

```
PS C:\DockerLabStudio> docker compose stop db
[+] Stopping 1/1
 ✔ Container lab-mysql-db  Stopped
PS C:\DockerLabStudio> Invoke-RestMethod 'http://127.0.0.1:5000/live' | Format-List
status  : alive
service : flask-web
PS C:\DockerLabStudio> Invoke-RestMethod 'http://127.0.0.1:5000/health' | Format-List
status   : degraded
database : unavailable
students :
service  : flask-web
message  : MySQL belum tersedia atau sedang dipulihkan.
PS C:\DockerLabStudio> docker compose start db
[+] Starting 1/1
 ✔ Container lab-mysql-db Started
```

Setelah health check MySQL pulih:

```
status   : ok
```

```
database : connected
service  : flask-web
```

G. NETWORK DAN DIAGNOSTIK (Gambar 2.7 / T07-T08)

```
PS C:\DockerLabStudio> docker compose exec -T web python -c "import socket; print('db=' + socket.gethostbyname('db'))"
db=172.20.0.2
PS C:\DockerLabStudio> docker compose exec -T web python -c "import os,mysql.connector;
c=mysql.connector.connect(host=os.getenv('DB_HOST','db'),port=int(os.getenv('DB_PORT','3306')),user=os.getenv('DB_USER'),password=os.g
etenv('DB_PASSWORD'),database=os.getenv('DB_NAME'),connection_timeout=4); c.close(); print('MYSQL_CONNECT_OK')"
MYSQL_CONNECT_OK
PS C:\DockerLabStudio> docker volume ls --filter "label=com.docker.compose.project=flask-mysql-docker-lab"
DRIVER    VOLUME NAME
local     flask-mysql-docker-lab_dbdata
PS C:\DockerLabStudio> docker network ls --filter "label=com.docker.compose.project=flask-mysql-docker-lab"
NETWORK ID     NAME                           DRIVER    SCOPE
a8b31e2c4d51   flask-mysql-docker-lab_appnet   bridge    local
```

H. VALIDASI INPUT (Gambar 2.8 / T10)

Kirim satu data dengan NIM unik melalui form POST. Kirim NIM sama lagi; respons halaman seharusnya menampilkan "NIM tersebut sudah terdaftar". Kirim nama dan NIM kosong; respons halaman seharusnya menampilkan "Nama dan NIM wajib diisi". Bersihkan record uji setelah verifikasi.

I. JALANKAN SEMUA TES DARI TERMINAL (Gambar 2.9)

```
PS C:\DockerLabStudio> .\uji-semua-tes.ps1
Docker Lab Studio - Test Center (ALL)
Run ID   : 20261020-231455
Mode     : ALL_NON_DESTRUCTIVE
Total    : 10
PASS     : 9
FAIL     : 0
SKIPPED  : 1
T01 PASS     Prasyarat & konfigurasi
T02 PASS     Build & startup service
T03 PASS     Create, read & delete mahasiswa
T04 PASS     Persistensi named volume
T05 PASS     Pemulihan MySQL
T06 SKIPPED  Reset volume database
T07 PASS     Network internal & koneksi DB
T08 PASS     Snapshot diagnostik
T09 PASS     Port & storage configuration
T10 PASS     Validasi input & NIM duplikat
Laporan JSON: reports\test-report-20261020-231455.json
Ekspor HTML/CSV/JSON juga tersedia di dashboard > Reports & evidence.
```

J. RESET VOLUME MANUAL (Gambar 2.10; OPSIONAL, TERPISAH DARI JALANKAN SEMUA TES)

PERINGATAN: perintah ini menghapus semua data di named volume database proyek.
Jalankan hanya setelah backup dan setelah bukti persistensi T04 disimpan.
Pilih salah satu cara: reset manual berikut ATAU jalankan T06 individual dari Test Center.
Jangan menjalankan keduanya berurutan kecuali memang ingin melakukan reset dua kali.

```
PS C:\DockerLabStudio> docker compose down -v
[+] Running 4/4
✔ Container lab-flask-web Removed
✔ Container lab-mysql-db Removed
✔ Network flask-mysql-docker-lab_appnet Removed
✔ Volume flask-mysql-docker-lab_dbdata Removed
PS C:\DockerLabStudio> docker compose up -d
[+] Running 4/4
✔ Network flask-mysql-docker-lab_appnet Created
✔ Volume "flask-mysql-docker-lab_dbdata" Created
✔ Container lab-mysql-db Started
✔ Container lab-flask-web Started
PS C:\DockerLabStudio> Invoke-RestMethod 'http://127.0.0.1:5000/health' | Format-List
```

status : ok
database : connected
students : 0
service : flask-web

Untuk menjalankan T06 otomatis melalui controller, gunakan:
PS C:\DockerLabStudio> .\uji-semua-tes.ps1 -TestId T06 -AllowDestructive
Ketik RESET untuk melanjutkan: RESET
Controller akan menjalankan down -v, membuat ulang stack, lalu memeriksa bahwa database kosong.

CATATAN AKHIR
Semua output sukses di atas adalah contoh yang diharapkan, bukan bukti eksekusi Docker aktual. Versi, waktu, durasi, IP, ID record, dan log dapat berbeda pada laptop.

Lampiran B — Batasan dan referensi

- Audit statis: hasil pemeriksaan berkas sumber, sintaks, konfigurasi, dan struktur paket.
- Runtime: Docker CLI/Engine dan Windows PowerShell tidak tersedia di lingkungan penyusunan paket; build/startup dan integrasi Flask–MySQL belum dieksekusi di sini.
- Screenshot web/terminal dengan status PASS adalah simulasi jalur yang diharapkan; caption gambar menjelaskan statusnya.
- Dokumentasi Docker Compose startup order: https://docs.docker.com/compose/how-tos/startup-order/
- Dokumentasi Docker volumes: https://docs.docker.com/engine/storage/volumes/
- Dokumentasi docker compose down: https://docs.docker.com/reference/cli/docker/compose/down/
- Dokumentasi Docker Desktop Windows: https://docs.docker.com/desktop/setup/install/windows-install/

Lampiran C — Membersihkan seluruh resource Docker proyek

Tujuan. Menghapus resource runtime Docker Lab Studio setelah praktikum agar proyek dapat dimulai kembali tanpa container, volume database, dan network lama.

Langkah penggunaan

8. Pastikan seluruh ZIP telah diekstrak ke satu folder dan Docker Desktop aktif.
9. Klik dua kali HAPUS-SELURUH-DATA-DOCKER.bat.
10. Baca peringatan lalu ketik HAPUS untuk menyetujui. Jika teks lain dimasukkan, operasi dibatalkan.
11. Tunggu perintah Docker Compose dan verifikasi akhir selesai.
12. Untuk praktik ulang, jalankan jalankan-sistem.bat.

Resource yang dihapus

- Container Compose milik proyek ini.
- Named volume proyek, termasuk seluruh data database MySQL.
- Network Compose milik proyek.
- Image aplikasi lokal yang dikelola Compose.
- Proses dashboard controller yang dijalankan dari folder proyek ini.

Resource yang dipertahankan

- Source code, compose.yaml, file .env, laporan, screenshot, dan dokumentasi.
- Docker Desktop, image dasar mysql:8.4, serta resource Docker milik proyek lain.

Peringatan: penghapusan volume bersifat destruktif. Data dalam volume database tidak dapat dipulihkan melalui Docker setelah volume dihapus. Skrip ini tidak menggunakan docker system prune atau perintah pembersihan global.